

Are you familiar with K8s?

It is another name for Kubernetes, an open-source platform for managing containerized applications. Kubernetes automates the scaling, deployment, and management of containerized applications, allowing them to run consistently across different computing environments.

What is Kubernetes and what does it do?

Kubernetes is an open-source container management tool that automates the deployment, scaling, and descaling of containers. It was developed by Google based on their experience of running containerized workloads for over 15 years. Kubernetes has become the standard tool for managing containerized applications and has a large and active community contributing to its development.

How are Kubernetes and Docker linked?

Docker builds containers, which then communicate with each other via Kubernetes. Kubernetes supports multiple container runtimes, including Docker, CRI-O, and others. In simple terms, Kubernetes is analogous to an operating system, and Docker containers are comparable to applications installed on that operating system.

Docker is a containerization platform that allows developers to package and distribute their applications as self-contained units, known as containers. Kubernetes, on the other hand, is a container orchestration platform that automates the deployment, scaling, and management of containerized applications.

The two technologies, Kubernetes and Docker, work together to enable the deployment and management of applications in a distributed environment.

Can you explain what container orchestration is?

Container orchestration is a process that involves managing and coordinating the deployment, scaling, and operation of multiple containers that run on a single server or across multiple servers. It automates container provisioning, networking, resource allocation, load balancing, availability, and lifecycle management tasks. Container orchestration tools, such as Kubernetes, Docker Swarm, and Apache Mesos, help ensure that containers work together seamlessly and efficiently to deliver the desired applications or services.

Why do we need container orchestration?

Container orchestration is critical to working with containers, allowing organizations to unlock their full benefits. It can be used in any environment where you use containers.

Container orchestration is needed to manage and automate containerized applications' deployment, scaling, and management. It helps to reduce operational overhead, increase efficiency and scalability, and ensure infrastructure availability, thus helping to improve application performance.

List the features of Kubernetes.

Some of the features of Kubernetes are:

Automated scheduling: Kubernetes automatically schedules containers to run on the available resources in the cluster.

Self-healing capabilities: Kubernetes automatically replaces failed containers and reschedules them on healthy nodes in the cluster.

Automated rollouts and rollback: Kubernetes is responsible for automating the deployment process, enabling users to roll out new versions of their applications easily and roll them back in case of any issues.

Horizontal scaling and load balancing: Kubernetes automatically scales and distributes traffic across multiple instances of a containerized application.

Configuration management: Kubernetes enables users to define and manage the application configuration settings separately from the application code.

Service discovery & networking: Kubernetes provides a built-in service discovery mechanism to locate and communicate with other services in the cluster.

Security & compliance: Kubernetes provides several security features, including access control to ensure the security of the cluster and compliance with organizational policies.

7. How does Kubernetes help in containerized deployment?

Kubernetes assists in containerized deployments by providing a platform for orchestrating, managing, and scaling containerized applications. It automates many of the manual processes involved in deploying, managing, and scaling containers, making it easier to deploy and manage complex, scalable systems. Kubernetes manages the lifecycle of containers, including starting, stopping, and restarting them, as well as distributing them across nodes in a cluster.

8. What are clusters in Kubernetes?

Clusters in Kubernetes refer to a group of interconnected physical or virtual machines called nodes that work together to host and manage containerized applications.

9. Explain Google Container Engine (Google Kubernetes Engine).

It is a Google-managed implementation of the Kubernetes open-source management platform for clusters and Docker containers. It provides a managed environment for deploying, scaling, and managing your containerized applications using Google infrastructure. It is designed to simplify containerized applications' deployment, management, and scaling in a production environment.

10.What is Heapster?

Heapster is a retired Kubernetes project designed for monitoring and analyzing container clusters. It served as a cluster-wide aggregator of monitoring and event data, running as a pod within the Kubernetes cluster.

11.What do you know about Minikube?

Minikube is a tool that allows you to run a single-node Kubernetes cluster locally on your machine, making it a great option for learning, experimenting, and developing applications on Kubernetes without needing a full-fledged cloud or multi-node setup.

12.What do you know about Kubectl?

In Kubernetes, kubectl is the primary command-line tool used to interact with the Kubernetes API server.

Key Functions of kubectl:

- **Managing Applications:**

Deploy, update, and scale applications by interacting with the Kubernetes API server.

- **Resource Management:**

Create, read, update, and delete (CRUD) operations on Kubernetes resources like pods, deployments, services, etc.

- **Cluster Inspection:**

View and monitor the status of the cluster, including nodes, pods, services, and other resources.

- **Troubleshooting:**

Debug applications by examining logs, attaching to containers, and creating debugging sessions.

- **Configuration Management:**

Apply and manage configuration files for Kubernetes resources.

How kubectl Works:

1. **1. Configuration:**

kubectl relies on a configuration file (kubeconfig) to connect to the Kubernetes API server. This file contains information about the cluster, including the API server address and authentication details.

2. **2. Communication:**

kubectl uses the Kubernetes API to interact with the API server, sending requests for operations like creating, reading, updating, and deleting resources.

3. **3. Resource Management:**

The API server processes these requests and interacts with the other components of the Kubernetes cluster (like the etcd data store, scheduler, and controller manager) to implement the desired changes.

Examples of kubectl Commands:

- `kubectl get pods`: Lists all pods in the cluster or a specified namespace.
- `kubectl deploy`: Creates a new deployment.
- `kubectl apply -f deployment.yaml`: Applies a configuration file to create or update resources.
- `kubectl logs <pod-name>`: Displays the logs of a specific pod.

In essence, kubectl is a powerful and versatile tool that empowers users to manage and control their Kubernetes clusters effectively.

14.What is a node in Kubernetes?

In Kubernetes, a node is a worker machine that runs containerized workloads (pods) as part of a cluster. It can be a physical machine or a virtual machine, and can be hosted on-premises or in the cloud.

Key Characteristics of Kubernetes Nodes:

- **Compute Resource:**

Nodes provide the physical or virtual resources (CPU, memory) for running pods.

- **Containerized Workloads:**

They host one or more pods, which are the smallest deployable units in Kubernetes.

- **Control Plane Management:**

The control plane, a separate component of the cluster, manages the nodes and their resources.

- **Automatic Scheduling:**

The control plane automatically distributes pods across the nodes in the cluster based on available resources.

- **Kubelet Agent:**

Each node runs a Kubelet agent, which communicates with the control plane, manages pods, and interacts with the container runtime.

- **Container Runtime:**

A container runtime is also present on each node to pull container images, unpack them, and run the application.

Nodes in a Kubernetes Cluster:

- **Worker Nodes:**

Nodes are often referred to as worker nodes because they perform the work of running pods.

- **Cluster Formation:**

Multiple nodes working together form a Kubernetes cluster, which provides high availability, scalability, and fault tolerance.

- **Scalability:**

Kubernetes can scale by adding or removing nodes, and it automatically redistributes workloads across the cluster.

- **Fault Tolerance:**

If a node fails, the cluster can automatically remove it and redistribute its pods to other healthy nodes.

In essence, nodes are the fundamental building blocks of a Kubernetes cluster, providing the hardware and software environment for running containerized applications.

15. List the main components of Kubernetes architecture.

Two main components of Kubernetes architecture are the Master node and the Worker node.

Master node: The master node is the control plane making global decisions inside the cluster. The master node comprises the control plane components responsible for managing and coordinating the cluster. These components are the API server, scheduler, cloud controller manager, and controller manager.

Worker node: The worker node has four very light components, which makes sense because you want to reserve most of the space for your pods. These components are the proxy, the Kubelet, and the container runtime.

16.Can you tell me about kube-proxy?

Kube-proxy is a network proxy that runs on each node in a Kubernetes cluster and manages network rules, enabling communication between pods and services.

Key Functions of Kube-proxy:

- **Traffic Routing:**

Kube-proxy forwards traffic from the service's IP address and port to the designated pods, ensuring that clients can reach the correct backend.

- **Service Management:**

It watches for changes to service and endpoint slices and updates the network rules accordingly.

- **Load Balancing:**

While not the primary load balancer, kube-proxy can contribute to load balancing by routing traffic across multiple pods.

- **Network Rules:**

Kube-proxy uses iptables or other mechanisms to configure network rules on each node, allowing traffic to be captured and redirected.

- **Integration with other components:**

Kube-proxy works in conjunction with other Kubernetes components like the API server and kubelet, ensuring seamless network communication.

- **Service Discovery:**

Kube-proxy helps in service discovery by enabling clients to find and connect to the appropriate service through its stable IP address.

17.What is the master node in Kubernetes?

In Kubernetes, master nodes, also known as control plane nodes, are responsible for managing and orchestrating the entire cluster. They make critical decisions about the cluster, including scheduling applications and maintaining the cluster's state. Worker nodes, on the other hand, are where the applications actually run.

18.Tell me more about the kube-scheduler.

The kube-scheduler is the default scheduler in Kubernetes responsible for assigning pods to nodes within a cluster. It runs as part of the control plane and analyzes nodes to find the best fit for newly created or unscheduled pods. The scheduler considers factors like resource availability, node constraints, and affinity/anti-affinity rules to make its decisions.

Here's a more detailed breakdown:

Function:

- **Node Selection:**

The kube-scheduler evaluates nodes in the cluster and selects the most suitable one for a given pod based on various criteria.

- **Resource Considerations:**

It considers resource requirements (CPU, memory, storage) of the pod and ensures that the selected node has enough available resources.

- **Constraint Handling:**

The scheduler also takes into account any user-defined constraints, such as node selectors, taints/tolerations, and affinity/anti-affinity rules.

- **Binding:**

Once a suitable node is found, the kube-scheduler binds the pod to that node, informing the API server about the decision.

Scheduling Process:

1. **Filtering:** The scheduler filters nodes that meet the basic requirements of the pod (e.g., resource requests, taints/tolerations).
2. **Prioritization:** It then ranks the remaining suitable nodes based on a set of configurable predicates and predicates.
3. **Binding:** Finally, the pod is bound to the node with the highest score.

Customization:

- While kube-scheduler is the default, Kubernetes allows for the creation and use of custom schedulers.
- This enables users to implement specialized scheduling logic based on their specific needs.

Example:

Imagine a pod requiring a specific GPU and running on a cluster with different types of nodes (some with GPUs, some without). The kube-scheduler would prioritize nodes with GPUs, ensuring the pod can run on the appropriate hardware.

19.Which node in Kubernetes keeps track of resource utilization?

The node in Kubernetes that keeps track of resource utilization is the kubelet. It runs on each worker node and is responsible for managing the state of the node, including starting and stopping pods, as well as monitoring their resource usage. The Metrics Server collects the relevant resource usage stats from kubelet and generates aggregated metrics via metrics API.

20.What is Kubernetes controller manager?

The Kubernetes controller manager embeds controllers and is responsible for creating a namespace. Here controllers are control loops that watch the state of your cluster, then make or request changes where needed. Every controller tries to move the current cluster state to the desired state.

21.List the different types of controllers in Kubernetes.

Here are some of the types of controllers:

- Node controller
- Replication controller
- Service account and token controller
- Endpoints controller
- Namespace controller

22.What do you know about ETCD?

Pronounced as "ett-see-dee," it is written in Go programming language and used to coordinate distributed work and key-value pairs. It is an open-source distributed key-value store that holds and manages the critical information distributed systems need to keep running. It is built on the Raft consensus algorithm, which ensures datastore consistency across all the nodes.

23. List the different types of services in Kubernetes.

Here are some of the services in Kubernetes:

Cluster IP: This is the default service type in Kubernetes, and it exposes the service on a cluster-internal IP. This means that only the services inside the cluster can access it.

Node Port: This type of service exposes the service on a static port on each node in the cluster. This makes the service accessible from outside the cluster.

Load balancer: This type of service provisions an external load balancer in the cloud infrastructure and directs traffic to the Kubernetes service. This allows you to expose your service to the internet.

External name: This type of service maps the service to an external DNS name. This allows you to reference external services by name from within your cluster.

24. Tell me about load balancers in Kubernetes.

In Kubernetes, a load balancer is a component that distributes incoming network traffic across multiple instances of an application running in a cluster. The load balancer sends connections to one server in the pool based on an algorithm to determine the next server and sends new connections to the next server, which is available. This algorithm is ideal where virtual machines incur costs, such as in hosted environments. Some of the strategies used for load balancing are Round robin, Session infinity, and IP hashing.

25.What is Ingress network?

A Kubernetes ingress is an API object that allows external traffic to be routed to the appropriate Kubernetes services based on the incoming request's URL or host. It is used to expose HTTP and HTTPS routes to the Kubernetes cluster.

26.What is the role of the Ingress network?

The Ingress network manages external access to services in the cluster. Specifically, it enables the load balancing of traffic, termination of SSL/TLS, and virtual hosting for HTTP and HTTPS routes exposed from outside the cluster to services within it. The Ingress network helps streamline communication between the applications and external clients by providing a single entry point for incoming traffic.

27.Explain the cloud controller manager.

The cloud controller manager lets you link your cluster into your cloud provider's API, and separates out the components that interact with that cloud platform from components that only interact with your cluster.

Advantages

- The CCM allows Kubernetes to work with different cloud providers (like AWS, Google Cloud, Azure) without being tied to a specific platform.
- Each cloud provider has its own CCM project, enabling Kubernetes to interact with their services (e.g., Elastic Load Balancers, Elastic Block Storage) without modification. In HA configurations, multiple instances of the CCM can run, ensuring that cloud interactions remain functional even if one instance fails.
- By automating the creation, deletion, and management of cloud resources, the CCM helps ensure optimal resource utilization within the Kubernetes cluster.

28.What are the different types of cloud controller managers?

Here are some of the types of cloud controller managers:

Node controller: Responsible for managing the lifecycle of nodes in the cluster, such as creating, updating, and deleting nodes

Route controller: Manages ingress and egress traffic for services running in the Kubernetes cluster

Volume controller: Responsible for managing the lifecycle of volumes in the cluster, such as creating, attaching, and detaching volumes.

Service controller: Manages the lifecycle of Kubernetes services, such as creating, updating, and deleting services.

29.Do you know what container resource monitoring is?

Container resource monitoring in Kubernetes involves tracking and managing the resources consumed by containers within a Kubernetes cluster. This helps in identifying resource bottlenecks, optimizing resource allocation, and ensuring the overall health and performance of applications. It includes monitoring CPU, memory, disk, and network usage

30.What is the init container?

An init container is a type of container in Kubernetes that runs before the main application containers in a pod. The purpose of an init container is to perform initialization tasks or setup procedures that are not present in the application container images. Examples of tasks that an init container might perform include downloading configuration files, setting up a network connection, or initializing a database schema.

Intermediate Kubernetes interview questions and answers

1.List some container resource monitoring tools.

Here are some of the resource monitoring tools:

- Grafana
- Kibana
- CAdvisor
- Prometheus
- SolarWinds
- ElasticSearch
- Sysdig

2.What is Grafana?

Grafana is an open-source interactive data visualization platform. It was developed by Grafana Labs, which allows users to see their data via charts, graphs, and alerts for the web when connected to supported data sources. It is divided into frontend and backend, written in TypeScript and Go, respectively.

3.What is CAdvisor?

cAdvisor is an open-source tool by Google used for monitoring the resource usage and performance of containers, particularly in Kubernetes environments. It collects, aggregates, and exports metrics like CPU, memory, and I/O usage for running containers, providing insights into their resource consumption and performance. In Kubernetes, cAdvisor is often integrated with the kubelet, a component that runs on each node and manages pods, making it a core part of the cluster's monitoring infrastructure.

Key Features and Functionality:

- **Real-time Monitoring:**

cAdvisor collects metrics in real-time, providing up-to-date information on container resource usage.

- **Resource Usage Metrics:**

It gathers data on CPU, memory, network, and file system usage, allowing for performance analysis.

- **Integration with Kubernetes:**

cAdvisor is designed to work seamlessly within a Kubernetes environment, particularly through its integration with the kubelet.

- **Open-Source and Versatile:**

It's an open-source tool that can be used in various containerized environments, from single Docker installations to full Kubernetes clusters.

- **Metrics Export:**

cAdvisor exports metrics in a Prometheus-compatible format, allowing for integration with monitoring and visualization tools like Prometheus and Grafana.

How cAdvisor Works in Kubernetes:

1. **Integration with Kubelet:**

cAdvisor is embedded within the kubelet, a component that runs on each Kubernetes node.

2. **Metric Collection:**

The kubelet uses cAdvisor to gather metrics about the containers running on that node.

3. **Metrics Exposure:**

cAdvisor exposes these metrics through an endpoint (e.g., /metrics/cadvisor) that can be scraped by Prometheus.

4. **Monitoring and Visualization:**

Prometheus scrapes the metrics and can store them in a time-series database. These metrics can then be visualized using tools like Grafana.

Benefits of using cAdvisor in Kubernetes:

- **Improved Resource Management:**

By monitoring container resource usage, you can better allocate resources to containers, optimizing performance and preventing resource contention.

- **Enhanced Performance Analysis:**

cAdvisor provides insights into container performance, allowing you to identify bottlenecks and areas for optimization.

- **Simplified Monitoring:**

cAdvisor simplifies monitoring by integrating directly with the Kubelet and exposing metrics in a standard format.

- **Scalability:**

cAdvisor is designed to handle large-scale Kubernetes clusters, providing reliable monitoring even with many containers.

4.What is Prometheus?

Prometheus is an open-source monitoring and alerting system of the Cloud Native Computing Foundation. It was originally developed by SoundCloud and is designed to help users monitor their systems and applications by collecting and processing real-time metrics data. Prometheus has a powerful query language and a flexible alerting system that allows users to define custom rules and receive alerts when certain conditions are met.

Additionally, Prometheus offers various options for data visualization, including built-in graph and dashboarding capabilities. Prometheus supports static and dynamic service discovery mechanisms to discover and monitor targets automatically.

5.Differentiate between a replica set and a replication controller.

They are the same but differ only in using selectors to reproduce pods. The replication controller allows us to create multiple pods easily, but if a pod crashes, it ensures it is replaced with a new pod. It can scale the number of pods and update or delete multiple pods with a single command.

The replica set is the same as the replication controller except that they have more options for the selectors. They use set-based selectors to manage the pods. Here the rolling-update command won't work.

6.Which selectors does the replica set use?

A replica set in Kubernetes uses label selectors to identify which pods it should manage. The selectors specify a set of key-value pairs that the replica set uses to match against the labels applied to the pods. Set-based selectors allow filtering keys according to a set of values. There are three kinds of operators: in, not in, and exists. The replica set will look for pods whose labels match the selectors.

7.Which selectors do replication controllers use?

Replication controllers use label selectors to identify the set of pods that they manage. Specifically, they use equality-based selectors, which allow filtering by label key and values. These selectors look for pods with labels that match a specific key-value pair. To use an equality-based selector, you can use the "-l" or "--selector" option.

8.What do equality-based selectors do?

They allow filtering by label keys and values. Thus they will only look for pods with the exact same phrase as the label. When a pod or other resource is created, it can be labeled with key-value pairs. Equality-based selectors allow you to select resources based on an exact match of those key-value pairs.

9.How do you monitor applications in Kubernetes?

Application monitoring with Kubernetes is not reliant on a single monitoring provider. You can utilize resource metrics or entire metrics pipelines to collect monitoring information on new clusters.

The resource metrics pipeline only delivers a subset of metrics related to cluster components such as the Horizontal Pod Autoscaler controller and the kubectl top function. The lightweight, short-term, in-memory metrics server

collects these measurements, which are then provided via the metrics.k8s.io API.

A complete metrics pipeline gives you access to more detailed metrics. Using technologies such as the Horizontal Pod Autoscaler, Kubernetes may respond to these indicators by dynamically scaling or adjusting the cluster based on its present condition.

10.What do you know about Headless service?

The Headless Service creates a service grouping that does not allocate an IP address or forward traffic. So here, the spec.clusterIP is explicitly set to "None," and the spec.type is set to "ClusterIP."

This service is used for stateful apps, such as databases, where it is essential to maintain a consistent network identity for each instance. With Headless service, you don't need the proxy to reach the pods.

11.List some security measures that you can take while using Kubernetes.

Here are some of the security measures we can take:

- Limit the access to ETCD
- Implement network segmentation
- Define source quota
- Provide limited access to nodes of Kubernetes

13.Explain orchestration when it comes to software and DevOps.

The automated configuration, coordination, and management of computer systems, services, and applications is called orchestration in DevOps. It leverages DevOps tools that allow for version control, rapid updates and releases, and other best practices for software engineering.

Orchestration in software involves coordinating and managing multiple computer systems, applications, and/or services, stringing together various tasks to execute a larger workflow or process. These processes can consist of numerous tasks that are automated and can involve multiple systems.

15.Tell the difference between Docker Swarm and Kubernetes.

Docker Swarm is a simpler, more straightforward solution for smaller deployments, while Kubernetes is a more complex, versatile, and scalable platform for larger, more complex applications.

It provides a wider range of features, including service discovery, load balancing, and more sophisticated networking options.

- **Choose Docker Swarm when:**

You need a simple, lightweight solution for smaller deployments, quick deployments, or learning container orchestration.

- **Choose Kubernetes when:**

You need a scalable, flexible, and feature-rich solution for large-scale, complex applications.

17.How can you use Kubernetes for workload distribution optimization?

Kubernetes enables efficient workload distribution optimization through several mechanisms, including scheduling policies, resource management, and autoscaling capabilities. By leveraging these features, Kubernetes can dynamically distribute workloads across nodes, ensuring optimal resource utilization and performance

1. Scheduling Policies:

- **Node Affinity and Anti-Affinity:**

Kubernetes allows you to specify which nodes a pod can or cannot be scheduled on, ensuring workloads are placed on suitable hardware or in specific regions.

- **Node Selectors:**

Using labels, you can filter nodes based on their characteristics (e.g., hardware, location) and direct pods to specific nodes.

- **Taints and Tolerations:**

Taints on nodes prevent pods without corresponding tolerations from being scheduled, allowing you to reserve resources or isolate specific workloads.

- **Topology-Aware Scheduling:**

For edge applications with latency requirements, Kubernetes can prioritize node placement based on proximity to data sources or end users.

2. Resource Management:

- **Resource Requests and Limits:**

Define the minimum and maximum resources (CPU, memory) a pod can consume, ensuring fair resource allocation and preventing runaway resource usage.

- **Resource Quotas:**

Set limits on resource usage per namespace, preventing any single workload from monopolizing resources in shared environments.

3. Autoscaling:

- **Horizontal Pod Autoscaling (HPA):**

Automatically scale the number of pod replicas based on resource utilization (CPU, memory) or custom metrics.

- **Vertical Pod Autoscaling (VPA):**

Dynamically adjust the resource requests and limits for each pod, optimizing resource allocation based on actual workload needs.

- **Cluster Autoscaler:**

Dynamically scales the number of nodes in your cluster based on workload demand, ensuring sufficient resources are available.

4. Observability and Monitoring:

- **Metrics Collection:**

Use tools like Prometheus and Grafana to collect real-time metrics on resource usage, helping you identify bottlenecks and tune configurations.

- **Logging and Tracing:**

Centralized logging and tracing provide insights into workload behavior, aiding in troubleshooting and optimization.

By leveraging these features, Kubernetes can effectively distribute workloads across a cluster, optimize resource utilization, and ensure high availability and performance.

19.What is contained in node status?

A node status contains address, condition, capacity, and info. It identifies the status of the nodes within the system.

Address: It depends on your cloud provider or bare metal configuration whether you have to use it or not.

Condition: It discusses the status or conditions of all running nodes.

Capacity: This block tells about the total resources processed by the node.

Info: It discusses general information about the node, such as the Kubernetes version, kernel version, container runtime details, and the current operating system which the node uses.

20. Tell the process name that runs on the Kubernetes master node.

The process that runs on the Kubernetes master node is the Kube-apiserver. This process is the access point for all the REST commands that control the cluster. It is the only Kubernetes control panel component with a user-accessible API and the sole master component you will interact with. It acts as a gateway to the cluster and supports life cycle orchestration.

23. What do you know about clusters of containers in Kubernetes?

The clusters of containers in Kubernetes are a set of nodes. They create some routes so the containers running on the nodes can communicate. They allow containers to run across multiple machines and environments, such as physical, virtual, and cloud-based. Kubernetes clusters coordinate processes such as scheduling and scaling applications, maintaining a cluster's state, and implementing updates.

24. Explain DaemonSets.

The DaemonSets are pod sets that run on a host only one time on a host. And so, they are used attributes of host layers such as -network or network monitoring. DaemonSet enables you to run the Kubernetes pod on the cluster nodes that fulfill certain criteria or requirements. It ensures that some or all nodes run a copy of a pod. DaemonSets are used to deploy background services, run cluster storage, log collection, and deploy a node monitoring daemon on every node

25. List some initial namespaces from which Kubernetes starts.

Some initial namespaces are as follows:

Kubernetes initially creates four default namespaces: default, kube-system, kube-public, and kube-node-lease.

- **default:** This namespace is where objects are placed if they don't specify a namespace.
- **kube-system:** This namespace is used for objects created by the Kubernetes system itself, including components and controllers.
- **kube-public:** This namespace is automatically created and is publicly readable, often used for cluster-wide resources that need public visibility.
- **kube-node-lease:** This namespace contains lease objects for each node, enabling the kubelet to send heartbeats for node failure detection.

26. What do you know about ClusterIP?

It is a default service in Kubernetes that apps inside your cluster can access. It provides a load-balanced IP address. One or more pods matching the label selector can forward traffic to the IP address.

With ClusterIP addresses, it is easier to manage and connect services in a Kubernetes cluster easier. Because each service has a unique IP address, traffic can be easily routed to the correct service without knowing the underlying pods' specific IP address.

27.What do you know about NodePort?

NodePort is a Kubernetes networking concept that allows external traffic to access a service in a cluster directly. It is done by opening a specific port on all nodes and sending traffic from this port to the service. When a service is created with a NodePort type, Kubernetes assigns a static port number to the service. The assigned port is then opened on every node in the cluster. This allows any traffic that is sent to this port on any node to be forwarded to the corresponding service.

30.Differentiate between ConfigMaps and Secret?

Where Secret stores critical data in an encrypted format, ConfigMaps stores the configuration of an application in a normal plain text format. However, both of them can be used as volume and be mounted on a pod through a pod definition file.

31.Explain Kubernetes RBAC.

It stands for Role-Based Access Control. It allows the management of permissions for users and groups inside a Kubernetes cluster. With RBAC, one can set access permissions and the required conditions to fulfill to gain access. Roles define a set of permissions granted to users or groups, and role bindings associate roles with specific users or groups.

Using RBAC, you can limit access to sensitive resources and ensure that only authorized users and applications can perform certain actions within the cluster. This helps to improve the security and reliability of your Kubernetes environment.

32.How can you get a static IP for a Kubernetes load balancer?

You can get a static IP for a Kubernetes load balancer by changing the DNS records. You must also check the Kubernetes version and allocate static IPs under Networking > External IP addresses. You can follow these steps for assigning a static IP:

- Reserve a static IP address from your cloud provider's pool of available IPs.
- Create a Kubernetes service of type "LoadBalancer."
- Assign the reserved static IP address to the service using the loadBalancerIP field in the service definition.
- Check that the service uses the static IP address by inspecting the service using the `kubectl describe service`.

33.What tools will you use for container orchestration?

Some tools we can use for container orchestration are:

Docker Swarm: It is an orchestration management tool running on Docker applications that helps in the creation and deployment of Docker nodes.

Apache Mesos: It is a cluster manager that helps in resource isolation and sharing.

Kubernetes: Kubernetes (commonly abbreviated K8s) is an open-source container orchestration system that automates software deployment, scaling, and management.

34.Can you list some of the objects of Kubernetes?

Here are some of the objects used by Kubernetes:

- Pods
- Controllers and replication set
- Cron jobs
- DaemonSets
- Distinctive identities
- Stateful sets
- Deployments

35.Define StatefulSets.

StatefulSets are a type of workload API that manage stateful applications. They can also be used to manage the scaling and deployment of pod sets. StatefulSets are often used to manage the deployment and scaling of pods that require stable network identities and persistent storage, making them well-suited for stateful workloads.

36.Can you list some reasons why DaemonSets are used?

DaemonSets are used for the following reasons:

Running system-level services: DaemonSets are often used to run system-level services that need to be deployed on every node in the cluster, such as logging agents, monitoring agents, or network services like load balancers.

Data collection: DaemonSets can collect data or metrics from each node in the cluster, such as system metrics, network traffic, or application logs.

Consistent deployment: By using DaemonSets to deploy services or applications, you can ensure that they are deployed consistently across all nodes in the cluster, which can help reduce configuration errors and ensure consistent operation.

High availability: Running services or applications as DaemonSets can provide high availability, as they can be automatically deployed on any new nodes added to the cluster and replaced if a node fails or becomes unavailable.

Resource isolation: Running services or applications as DaemonSets can provide resource isolation, as they can be deployed on specific cluster nodes with the necessary resources or configurations, such as specialized hardware or network connections.

37.What is the purpose of Operators?

In Kubernetes, managing applications isn't as simple as maintaining stateless applications where each replica reaches the required status and updates are done the same way. Because of the stateful nature of stateful applications, upgrading each replica may necessitate distinct treatment. As a result, managing stateful applications frequently necessitates using a human operator. This is where Kubernetes Operator is useful.

Kubernetes Operators are software extensions that automate the management of complex applications on Kubernetes clusters. Operators are designed to be highly customizable and adaptable to the needs of specific applications. They can perform tasks such as monitoring the health of an application, performing backups and restores, upgrading or rolling back versions, scaling resources up or down, and more.

38.Can you tell me some Kubectl commands?

Some Kubectl commands are:

- kubectl annotate
- kubectl config
- kubectl autoscale
- kubectl config current-context
- kubectl apply
- kubectl config set.
- kubectl cluster-info
- kubectl attach

39. Why is the Kube-apiserver used?

The kube-apiserver in Kubernetes serves as the central point of communication and control for the cluster, acting as the single source of truth for all cluster state and configuration. It validates and configures data for various API objects like pods, services, and replication controllers, ensuring consistency and integrity. Essentially, it's the gateway through which users, other Kubernetes components, and external systems interact with the cluster.

Here's a more detailed explanation of its role:

- **Centralized Communication:**

The kube-apiserver exposes the Kubernetes API, a RESTful interface, allowing users to interact with the cluster using tools like kubectl or directly through HTTP requests. It handles authentication, authorization, and access control for all requests.

- **State Management:**

The API server manages and stores the cluster's state in the etcd data store, ensuring consistency and durability. It's the only component that directly interacts with etcd.

- **Object Manipulation:**

It enables users to create, update, and delete Kubernetes objects like pods, services, deployments, etc., through the API.

- **Validation and Configuration:**

The API server validates the requests and configurations against the Kubernetes API schema, ensuring that the cluster's state remains consistent and adheres to defined policies.

- **Controller Communication:**

The kube-apiserver facilitates communication between the controllers (e.g., kube-scheduler, kube-controller-manager) and the cluster's resources, allowing them to manage the cluster's state based on the desired configurations.

Syntax:

kube-apiserver [flags]

Advanced Kubernetes interview questions and answers

1.What you will do to upgrade a Kubernetes cluster?

To upgrade a Kubernetes cluster, you will typically follow a process that involves upgrading the control plane first, then upgrading the worker nodes, and finally, updating tools like kubectl.

Here's a more detailed breakdown of the process:

1. Backup and Planning:

- **Backup:**Before starting the upgrade, it's crucial to back up your cluster using tools like Velero to ensure you can roll back if needed.
- **Planning:** Review the release notes for both Kubernetes and your specific cloud provider (e.g., EKS, AKS, GKE) to understand the upgrade process, compatibility issues, and any required steps.
- **Check Current Status:** Verify that your cluster is in a healthy state before proceeding with the upgrade.

2. Upgrading the Control Plane:

- **Upgrade Kubeadm:** Use the kubeadm tool to upgrade the kubeadm tool itself.
- **Upgrade Control Plane Components:** Upgrade the control plane components (API server, scheduler, controller manager) using kubeadm upgrade commands.
- **Verify Upgrade:** Confirm that the control plane has been successfully upgraded and is running the new version of Kubernetes.

3. Upgrading Worker Nodes:

- **Drain Nodes:** Drain the nodes (evict pods) that are about to be upgraded to minimize disruption during the upgrade.
- **Upgrade Kubeadm:** Upgrade the kubeadm tool on the worker nodes.
- **Upgrade Kubelet and Kubectl:** Update the kubelet and kubectl on the worker nodes.
- **Uncordon Nodes:** Once the upgrade is complete, uncordon the nodes to allow pods to be rescheduled.
- **Verify Upgrade:** Ensure the worker nodes are running the new Kubernetes version.

4. Post-Upgrade Steps:

- **Update Manifests:** Adjust any manifests or configuration files that may need to be updated due to changes in the Kubernetes API.
- **Monitor:** Monitor the cluster and applications during and after the upgrade to ensure everything is working as expected.
- **Cleanup:** Clean up any temporary resources or files created during the upgrade process.

Important Considerations:

- **Rolling Upgrades:** Consider using rolling upgrades to minimize downtime during the upgrade process.
- **Test Environment:**

Test the upgrade process in a staging environment before applying it to production.

- **Compatibility:** Ensure that your add-ons, custom controllers, and workloads are compatible with the new Kubernetes version.

- **Automated Testing:** Use automated testing frameworks like Helm tests or custom e2e tests to validate upgrades.
- **Feature Gates:** Leverage feature gates to enable or disable new features progressively, aiding in smoother transitions.

2.Explain the two types of Kubernetes pods.

The two Kubernetes pods are single-container pods and multi-container pods. Here's a brief explanation of each.

Single-container pods: These pods contain only one container and are the most common type of pod used in Kubernetes. They can be created using commands such as `kubectl run` or `kubectl create`.

Multi-container pods: These pods contain multiple containers that are tightly coupled and need to run together on the same host. Multi-container pods are created using the `kubectl create` command with a YAML file that defines the pod's configuration..

3.Tell me about labels in Kubernetes.

Labels are added to objects during their creation and can be altered by the users at run time. They are precisely collections of keys that hold some values.

They are a type of metadata that work through the concept of key-value pair attached to objects. In many cases, they are also used to define "identifying aspects" of an object mainly to help the user at a later stage. Also, as they do not imply semantics to Kubernetes by default, they do not directly change any functionality like other metadata.

4.List some objectives of the replication controller.

Some of the objectives of a replication controller are:

- Control and administer pod lifecycle

- Monitor and verify the permissible number of replicas
- Help user to check pod status
- Enable the alteration of pod

5.What is persistent volume?

Persistent volume (PV) is a storage piece that an administrator provides to a Kubernetes cluster. Suppose a developer needs persistent storage for an application in a cluster; they do so by creating a persistent volume claim (PVC) and then mounting the volume to a path in the pod. Once this process is done, the pod can claim any volume matching its requirements.

6.Do you know anything about the Secrets in Kubernetes?

Yes, the Secrets in Kubernetes are objects that hold sensitive information such as login credentials for the user, passwords, OAuth tokens, and SSH keys. Here “Secrets” give you more control over this information and its accidental exposure. Using Secrets, Kubernetes ensures that the sensitive data is encrypted, only authorized entities can access it, and it is not exposed to unauthorized users or applications.

9.Mention some differences between Kubernetes Volumes and Docker Volumes .

Kubernetes Volumes	Docker Volumes
Not limited to any container	Limited to the pod in a container
Support all containers deployed in the pod	Does not support all the containers deployed
The Kubernetes cluster manages Kubernetes volumes.	The Docker engine manages Docker volumes
Kubernetes volumes are tied to the pod's lifecycle, which means that they are created when the pod is created and destroyed when the pod is terminated.	Docker volumes have a lifecycle that is independent of the container that uses them, which means that they can persist even after the container is removed.

10.How can you provide API-Security on Kubernetes?

Securing APIs on Kubernetes is crucial to prevent unauthorized access and data breaches. One approach is to implement Role-Based Access Control (RBAC) policies with the kube-dashboard, which allows defining permissions for individual users or groups. Another option is to use Kubeless, a Kubernetes-native serverless framework, to protect APIs by automatically injecting security measures such as rate limiting and authentication. Additionally, it's important to use the correct authorization mode, such as the Node Authorizer or Attribute-Based Access Control (ABAC), to ensure that only authorized requests are accepted. By combining these methods, Kubernetes can provide strong API security for your applications.

11.How do you debug a POD that isn't being scheduled?

The scheduler in K8 is in charge of spawning pods into nodes. A variety of circumstances can cause unstartable POD. The most typical reason for POD not starting is a lack of resources; use commands like `kubectl describe POD -n Namespace` to find out why. Also, watch `kubectl get events` to observe the cluster's events.

12. List some of the types of Kubernetes volumes.

The different types of Kubernetes volumes are as follows:

EmptyDir: This volume is first created when a node is assigned with a pod. Initially, it is empty. A volume of type emptyDir is available for the lifetime of the pod.

Flocker: It is an open-source and clustered container data volume manager.

HostPath: This volume mounts a file or directory from the host node's filesystem into the pod. It can provide access to host files or share files between containers on the same host.

NFS: Network File System (NFS) allows computers to either access or share files over the network. It is a dedicated file storage when multiple users must retrieve data for centralized disk capacity.

13. What is PVC?

PVC stands for Persistent Volume Claim, a request made by a user for storage in a Kubernetes cluster. When a user creates a PVC, Kubernetes matches it with an appropriate Persistent Volume (PV) based on the storage requirements and properties specified in the PVC.

Persistent Volumes are network-attached storage devices that multiple pods can access in a cluster. A PVC is a request for a specific amount of storage, which is bound to a single pod or multiple pods, depending on the user's requirements.

14.What do you know about Kubernetes network policy?

The Kubernetes network policy indicates how pods in the same namespace communicate with one another and the network endpoint. Network policies allow you to control traffic at the IP address and port level. With Kubernetes network policy, you can define rules that specify which traffic can flow between pods and which traffic should be blocked.

15.What are the limitations of using the default namespace?

Using the default namespace alone makes it difficult to keep track of all the applications you can manage in your cluster over time. Custom namespaces help in grouping apps into logical groupings, such as one namespace for monitoring programs and another for security applications, and so on. This makes it easier to manage and monitor different applications within your cluster and control access to specific resources.

17.How can you perform maintenance in a single pod?

Here are the steps for performing maintenance in a single pod:

- Get the name of the pod you want to perform maintenance using the command
- Put the pod in maintenance mode by adding a label to it. You can use any label name, but here we'll use "maintenance-mode."
- Verify that the label has been applied to the pod
- Perform maintenance on the pod as needed
- Remove the maintenance label from the pod when you're done
- Verify that the label has been removed

18.How can you control the usage of resources in a pod?

We can control the resource usage of the pod through the use of limit and request.

Request: It controls the minimum guaranteed amount of resources (CPU and memory) that are required for a container.

Limit: It puts a limit on the resources any single container can make use of. This means that the container can never consume more than the memory amount or CPU amount indicated.

You can think of it as request is the minimum amount of resources to be used by a pod while limit is the maximum amount of resources that pod can use.

19.List various K8's services that can run on nodes.

Kubernetes (K8s) runs various services on nodes, including:

On Worker Nodes:

kubelet: Responsible for node management and ensuring that containers are running as expected.

kube-proxy: Provides network proxy services to enable communication between nodes and services.

On Master Nodes:

kube-apiserver: Provides the Kubernetes API through which all the other components interact with the cluster.

kube-scheduler: Assigns newly-created pods to a node based on resource availability and other constraints.

kube-controller-manager: Monitors the cluster state and performs tasks to maintain desired state, such as scaling up or down.

20.What is PDB (Pod Disruption Budget)?

The PDB (Pod Disruption Budget) allows you to limit the disruption to application use when its pods need to be rescheduled for some other reason on the Kubernetes nodes.

Kubernetes' Pod Disruption Budget (PDB) is a handy layer of defense for dealing with this type of issue. If you already have a Kubernetes cluster in production, the command 'kubectl get pdb --all-namespaces' will show you all the PDBs in all the namespaces of your Kubernetes cluster.

A Kubernetes administrator can make a one-of-a-kind deployment. Pod Disruption Budget ensures that the application's minimum number of running pods is preserved, as specified by the property minAvailable in the spec file. It is useful while executing a drain, as the drain will stop until the PDB is respected, ensuring the application's High Availability (HA). In the following spec file, minAvailable is set to 2, implying that the minimum number of available pods is two (even after the election).

21.Why should we use custom namespaces ?

If you use default namespaces alone, it gets difficult in the long run since you will not be able to manage them within a cluster as a whole. By default, all resources in a Kubernetes cluster are created in the default namespace, but as your application grows and you add more resources, it can become difficult to manage them all in one namespace.

With custom namespaces, this problem is resolved. By creating custom namespaces, you can logically group your resources based on your needs, such as separating production and development environments or separating applications by team or department. This makes it easier to manage and maintain your resources within the cluster, and also provides better security and resource isolation.

22.What are the various logging patterns you can use to get central logs from pod?

There are some common logging patterns as follows:

Node level logging agent: This involves installing a logging agent directly on the node where the pod is running. The logging agent collects logs from all the containers on that node and sends them to a central logging server.

Streaming sidecar container: In this pattern, a separate container, known as a sidecar, is added to the pod. The sidecar container is responsible for streaming the logs from the other containers in the pod to a central logging server.

Sidecar container with the logging agent: Similar to the streaming sidecar container pattern, a separate container is added to the pod. However, in this pattern, the sidecar container also includes a logging agent that collects logs from the other containers in the pod and streams them to a central logging server.

Fluentd logging pattern: This pattern involves deploying Fluentd, a unified logging layer, to capture, filter, and forward logs generated by the main container to a central logging system.

23.In the service given below, how can you convert the spec into an external one?

```
spec:
  selector:
    app: some-app
  ports:
    - protocol: UDP
      port: 9090
      targetPort: 9080
```

By using LoadBalancer as follows

```
spec:
  selector:
    app: some-app
  type: LoadBalancer
  ports:
    - protocol: UDP
      port: 9090
      targetPort: 9080
```

The type LoadBalancer field creates an external load balancer to distribute traffic to the service across the worker nodes.

24. Complete the following configuration spec file and turn it Ingress:

```
metadata:
  name: someapp-ingress
Spec:
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: someapp-ingress
spec:
  rules:
    - host: my.host
      http:
        paths:
          - backend:
              serviceName: someapp-internal-service
              servicePort: 9090
```

25. Can you schedule the pods to the node if the node is tainted?

If a node is tainted, pods will not be scheduled on it by default, but you can use tolerations in the pod spec to allow specific pods to be scheduled on the tainted node. Tolerations are used to specify that a pod can tolerate (or "ignore") a certain taint, allowing it to be scheduled on a tainted node. This can be useful in scenarios where you want to reserve certain nodes for specific types of workloads or to mark nodes as unsuitable for certain workloads.

26.How can you achieve zero downtime in Kubernetes?

We can achieve zero downtime in Kubernetes through RollingUpdate strategy. It is a process that allows updating the Kubernetes system with little effect on performance and zero downtime. This strategy involves gradually replacing old instances of an application with new ones, thus ensuring that the application is always available to end-users.

When you use RollingUpdate strategy, Kubernetes creates a new replica set with the updated version of your application and gradually replaces the old replica set with the new one. This ensures that the new version is rolled out to users gradually and any issues can be caught early.

27.How can you check that a pod always runs?

We can check that a pod always runs through a liveness probe. In case the check fails, it restarts the container. This probe is helpful in multiple instances when the container runs but the application crashes.

Example:

```
spec:
  containers:
  - name: liveness
    image: k8s.gcr.io/liveness
    args:
    - /server
    livenessProbe:
      httpGet:
        path: /heal
```

In this example, the "/heal" endpoint of the "k8s.gcr.io/liveness" container is checked by the liveness probe. If the probe fails to receive a response from this endpoint, Kubernetes will automatically restart the container. This helps ensure that the pod always runs, even if the container encounters issues.

28.Can you tell if the rolling update with stateful set replicas = 1 looks right?

No, it doesn't look right since there is only 1 replica, and any changes to the state full set may result in an outage. It means that the rolling update of a StatefulSet needs to rip apart one or more old pods before replicating them. Therefore, it's generally recommended to have at least two replicas in a StatefulSet to ensure high availability during rolling updates.

29.Suppose a pod exceeds its memory limit. What signal will be sent to the process?

In such cases, SIGKILL instantly ends the container and spawns a new one using out of memory (OOM) error. This will result in the kernel sending a SIGKILL signal to the process running inside the container. By default, Kubernetes will send a SIGTERM signal to the container's main process and wait for a certain period of time (determined by the terminationGracePeriodSeconds parameter) for the process to exit gracefully.

30.How can you run a pod on a specific node?

We can run a pod on a specific node using node affinity. Here, a node is assigned to an arbitrary label, and they are configured to be assigned to that node as per the label created.

For example, this code snippet creates an arbitrary label, node location, and assigns the value Germany to the node named person-01:

kubectl label nodes person-01 nodelocation=Germany

31.What happens when the master or the worker node fails?

Inside Kubernetes, when the master node fails, the cluster remains operational. There is no effect on the pod creation or any service member changes.

If a worker node fails, the master stops receiving updates, and any pods running on the failed worker node are terminated. The Kubernetes scheduler will then create new replicas of the terminated pods on other healthy worker nodes in the cluster.